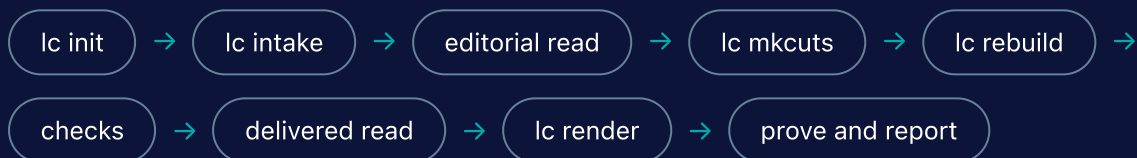


lesson-cut

Cut a long recorded lesson down to a tight, captioned video that reads as one clean take, with every check run before the render.



1 h+ → under ½

a real take of over an hour, cut to under half its length

~200 cuts

retakes, repeats, fillers, dead air

6 / 6

QC passes on the finished file

Contents

1	What it does
2	How to read this guide
3	Install
4	The workflow at a glance
5	A first edit, step by step
6	The editorial method
7	The editorial read, with agents
8	Anatomy of a project
9	Configuration reference
10	Command reference
11	Tool reference
12	The checks, and what each one alone can see
13	Troubleshooting
14	Operating contract for agents
15	Limits and known gaps
16	How it was verified
17	Maintaining and sharing it
18	Glossary

New to it?

Read chapters 3, 4 and 5, then 6.

An agent operating it?

Chapter 14 first, then 4, 6, 7 and 12.

Something went wrong?

Chapter 13.

Changing or sharing it?

Chapters 15 to 17.

1 What it does

You hand it a raw take: 30 to 90 minutes of someone building or explaining something live, usually a screen recording with a voice, sometimes a camera piece. It hands back a finished video that says everything the take said, with none of the stumbles, and burned-in captions.

Concretely, it removes:

- retakes and abandoned attempts, keeping the last complete one
- repeated sentences and repeated sections
- spoken instructions to the editor ("delete this", «احذف هاي») together with what they refer to
- failed on-screen actions and their retries (a dictation that never landed, a click that did nothing)
- truncated words, doubled words, stutters and standalone fillers ("OK", «تمام», "um")
- wordless gaps between words, trimmed to a short beat, while keeping a glimpse of the screen during long waits (the silence before the first word and after the last is a head and a tail cut)

It is an extraction of a working pipeline used on a paid Arabic course. There, a take of well over an hour came out at under half its length, and the finished file passed all six passes of its QC gate.

It is three things working together:

- **A method** (the `lesson-cut` skill): how to decide what to cut, written down precisely enough that an agent can apply it. This is the part that makes the result feel clean.
- **Agents** that carry the method out at scale: a window reader, an adversarial verifier and a completeness critic.
- **A toolchain** of about two dozen scripts behind one command, `lc`, that does the mechanical work reliably: it places every cut edge in silence, never inside a word, and it refuses to render until a set of checks is clean.

It does not upload or publish the lesson, and it does not decide for you. It stops at a rendered, checked file and a report. (With the ElevenLabs backend, the take's audio is sent to ElevenLabs for transcription; with the local backend nothing leaves the machine.)

2 How to read this guide

- **Using it for the first time:** sections 3, 4 and 5, then 6.
- **An agent operating the plugin:** section 14 first, then 4, 6, 7 and 12, with 9 to 11 as reference.
- **Something went wrong:** section 13.
- **Changing or sharing it:** sections 15 to 17.

Commands are written as `lc ...`. The real path is `<plugin>/scripts/lc`: in a terminal after the GitHub install, `~/claude/plugins/marketplaces/lesson-cut/scripts/lc`; in your own clone, `~/lesson-cut/scripts/lc` (section 3). The plugin's skill and commands write it as

`${CLAUDE_PLUGIN_ROOT}/scripts/lc`, a placeholder Claude Code fills in when it loads the plugin's own files. It is not an environment variable in shell commands, so every shell command carries the full path (section 14, rule 2).

3 Install

What the machine needs

Requirement	Why	Check
ffmpeg and ffprobe, built with libass	cutting, measuring, captions, rendering	<code>lc doctor</code>
Python 3.9 or newer	every tool	<code>lc doctor</code>
numpy, pillow, requests	installed into a private venv by <code>setup.sh</code>	<code>lc doctor</code>
A transcript source	an ElevenLabs key, or local whisper	<code>lc doctor</code>
The caption font	captions are burned into the picture	<code>lc doctor</code> asks libass directly
Disk	a half-hour 4K screen recording uses about 3 GB of render scratch plus the take; camera footage needs more	<code>lc doctor</code> prints free space

On macOS, install `brew install ffmpeg-full`: Homebrew's plain `ffmpeg` formula is built without libass and cannot burn captions (if both are installed, `brew unlink ffmpeg && brew link ffmpeg-full`). `lc doctor` checks for it. On Debian or Ubuntu, `sudo apt install ffmpeg python3-venv` covers ffmpeg (that package has libass) and the venv module `setup.sh` needs. The plugin was verified on macOS with Python 3.9.6; Linux should work (the shell scripts avoid BSD-only flags) but has not been run.

The default caption font is Almarai ExtraBold, free under the SIL Open Font License from <https://fonts.google.com/specimen/Almarai>. On macOS, open the .ttf files and click Install; on Linux, copy them to `~/.local/share/fonts` and run `fc-cache -f`. Any font you set in `captions.font` (section 9) works instead, as long as libass can find it.

From GitHub

In Claude Code:

```
/plugin marketplace add aiwithmo/lesson-cut
/plugin install lesson-cut@lesson-cut
```

Then, in a terminal, the one-time setup:

```
bash ~/.claude/plugins/marketplaces/lesson-cut/scripts/setup.sh # builds ~/.lesson-cut/venv, then runs the doctor
```

Then run `/lesson-doctor` in Claude Code.

The first command makes Claude Code keep a git clone of the repository at `~/.claude/plugins/marketplaces/lesson-cut` (the folder is named after the marketplace; if `CLAUDE_CONFIG_DIR` is set, it lives under that directory instead of `~/.claude`). That clone is the stable path for running the tools by hand:

```
LC=~/.claude/plugins/marketplaces/lesson-cut/scripts/lc
```

The copy Claude Code actually runs is a separate one, under `~/.claude/plugins/cache/lesson-cut/lesson-cut/<version>/`. Never point an alias or a symlink there: the path changes with every version, and the old copy is orphaned on update, so a link keeps running old code and then breaks.

From a clone

For contributors, or anyone who wants the repository in a folder of their own:

```
git clone https://github.com/aiwithmo/lesson-cut ~/lesson-cut
bash ~/lesson-cut/scripts/setup.sh # builds ~/.lesson-cut/venv, then runs the doctor
```

Then in Claude Code:

```
/plugin marketplace add ~/lesson-cut
/plugin install lesson-cut@lesson-cut
```

Here `lc` is `~/.lesson-cut/scripts/lc`. Claude Code keeps referring to `~/.lesson-cut`, so leave the folder where it is. If you move it, run the marketplace line again with the new path. Installing from a folder copies the whole folder into Claude Code's plugin cache, untracked files included, so keep keys, takes and renders out of it.

`setup.sh` puts its virtualenv in `~/.lesson-cut/venv`, outside the plugin, on purpose: Claude Code copies an installed plugin into its own cache, and a virtualenv does not survive being copied. Every entry point looks in `~/.lesson-cut`, so the marketplace clone, your own clone and the installed copy all share one environment. Set `LESSON_CUT_HOME` to put it somewhere else.

Updating

In Claude Code:

```
/plugin marketplace update lesson-cut
/plugin update lesson-cut@lesson-cut
```

Then restart Claude Code. The first line only refreshes the marketplace's copy of the repository; the second installs the new version into Claude Code's cache, and the restart loads it. In your own clone, run `git -C ~/lesson-cut pull` first, then the same two lines. Running `setup.sh` again after an update is safe and picks up any new requirement. `CHANGELOG.md` says what each version changed.

Installed copies are kept by version, so `/plugin update` brings in a change only when the version number changed. The marketplace clone, on the other hand, follows the repository after the first line even when the version did not change, so a by-hand `lc` from that clone can be newer than the plugin Claude Code runs.

A project whose captions were built before 1.2.0 needs `lc build_captions .` (or `lc rebuild`) before its next render. Older versions wrapped every caption line in right-to-left embedding, and the QC gate now fails a wrapped line with no right-to-left script. `lc render` refuses such a file before it starts, so the cost is seconds, not a render.

Moving from an older install

Versions before 1.2.0 were installed from a zip, as a folder marketplace (usually `~/lesson-cut`). Claude Code will not switch a marketplace that came from a folder over to GitHub in place: the add command refuses, because the source differs. Remove it first, then install as above:

```
/plugin marketplace remove lesson-cut
/plugin marketplace add aiwithmo/lesson-cut
/plugin install lesson-cut@lesson-cut
```

Removing the marketplace also uninstalls the plugin. `~/.lesson-cut`, with its venv and key file, is left alone, so the new install uses them as they are. By hand, `lc` is now the marketplace path above. When the new install works (`/lesson-doctor`), the old `~/lesson-cut` folder can be deleted. Rebuild the captions of any project in progress before its next render, as described under Updating.

Transcription: a key, or local

Pick one.

ElevenLabs Scribe (paid, accurate, fast: about a minute for a take over an hour long). Put the key in the environment, or in a file outside the plugin:

```
(umask 077; mkdir -p ~/.lesson-cut && echo 'ELEVENLABS_API_KEY=your-key-here' > ~/.lesson-cut/env)
```

Each take's audio is uploaded to ElevenLabs. The plugin reads the key from `ELEVENLABS_API_KEY` first, then from `~/.lesson-cut/env` (`$LESSON_CUT_HOME/env` when that is set), and from nowhere else. It never reads a key from inside the plugin or a project, because a key there is one `git add` away from being published.

Local whisper (free and private, but slow on a laptop CPU: the small model took several times the length of the clip on a busy machine, and large-v3 is slower still):

```
bash ~/.claude/plugins/marketplaces/lesson-cut/scripts/setup.sh --local-asr # installs faster-whisper
into the venv
```

(In your own clone: `bash ~/lesson-cut/scripts/setup.sh --local-asr`.)

Nothing leaves the machine. The model weights download on first use, and the model decides whether the transcript is usable at all. Measured on mixed Arabic and English speech against a paid transcript of the same 60 seconds:

Model	Result
base	unusable: 0 of 41 words right when forced to Arabic; every Arabic word dropped on auto-detect
small	36 of 41 words, starts within 0.17s on average: good enough to place cuts
large-v3 (the default)	the most accurate and the slowest; the first run downloads about 3 GB

So `lc intake --backend local --model small` is the practical floor for Arabic, and `large-v3` is worth its download for a lesson that ships. Every Latin token still gets checked against the screen before it reaches a caption, which is the rule anyway.

Until one of the two exists, `lc doctor` says "not ready". That is expected.

The doctor

Real output of version 1.2.0, run by hand from the marketplace path the GitHub install uses, on a machine with the environment built and no transcription backend chosen yet. Only the home directory has been changed, to `/Users/you`:

```
$ lc doctor
lesson-cut doctor
plugin: /Users/you/.claude/plugins/marketplaces/lesson-cut

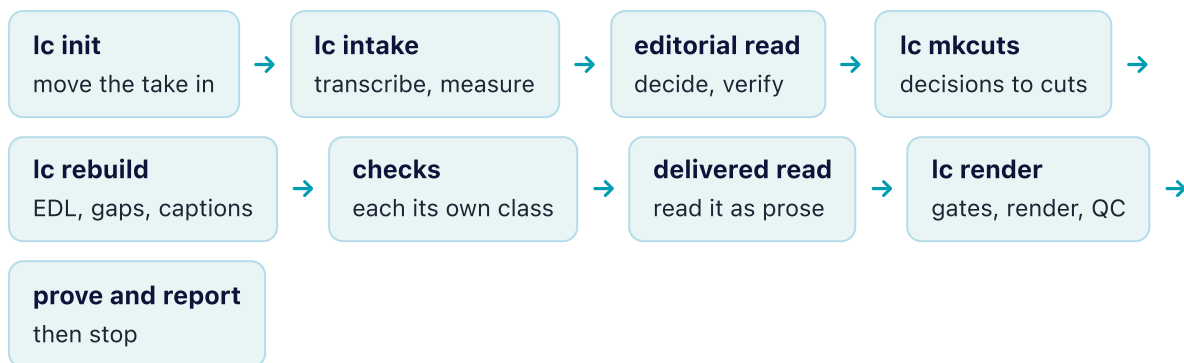
ok      ffmpeg (ffmpeg version 8.1.2)
ok      ffprobe (ffprobe version 8.1.2)
ok      ffmpeg has libass (the ass filter that burns captions)
ok      python 3.9 (/Users/you/.lesson-cut/venv/bin/python)
ok      python module numpy
ok      python module PIL
ok      python module requests
MISSING no ASR backend.
        Either: (umask 077; mkdir -p "/Users/you/.lesson-cut" && echo 'ELEVENLABS_API_KEY=sk_...' >
"/Users/you/.lesson-cut/env")
        Or:      bash "/Users/you/.claude/plugins/marketplaces/lesson-cut/scripts/setup.sh" --local-asr
(local, free, slower)
ok      font: libass resolves 'Almarai ExtraBold' -> /Users/you/Library/Fonts/Almarai-ExtraBold.ttf, -1,
Almarai-ExtraBold
note    free disk here: 14Gi (a half-hour 4K screen recording uses about 3 GB of render scratch; camera
footage needs more)

not ready: fix the MISSING lines above.
```

Run as `/lesson-doctor` inside Claude Code, the doctor runs from the installed copy, so its own paths (the `plugin:` line and the setup command) point into `~/.claude/plugins/cache/` instead.

The font line is the one to trust most. It asks libass which file it actually resolved, because a missing font does not fail anything: libass silently substitutes another face, and the first sign would otherwise be the delivered pixels after a long render. Run inside a project (its directory or its `edit/`), the doctor checks that project's caption font.

4 The workflow at a glance



Phase	Command	What happens
1. Intake	<code>lc init</code> , <code>lc intake</code>	Move the take in, probe it, transcribe once, measure every word
2. Orient	(reading)	Read the script, look at the picture, estimate the length, check lip sync
3. Editorial read	agents, <code>lc words</code>	Decide every cut against the script, then try to refute each one
4. Author the cut	<code>lc mkcuts</code> , <code>lc rebuild</code>	Decisions become the cut list; the cut, gaps and captions are rebuilt
5. Checks	<code>lc post_cut_check</code> and others	Each check sees one class of defect no other can
6. Delivered read	<code>lc delivered_read .</code>	Read the finished cut as prose, joins marked
7. Render and prove	<code>lc render</code>	Gates, render, QC gate; then look at the file yourself
8. Report	(writing)	What was cut, what was fixed, what only the operator can decide. Stop.

5 A first edit, step by step

Every output below is real, from one continuous run of version 1.1.1 on a 25 second bilingual clip (an English dictation, then Arabic). The transcript is the one ElevenLabs Scribe returned for that clip; where a block leaves lines out, it says `...`. Two things were changed after the run: the home directory is shown as `/Users/you`, and the project is called `demo`.

1. Make a project. The source is moved in, not copied.

```
$ lc init demo --source ~/Downloads/export.mp4 --lang ar
-- source: /Users/you/demo/take1.mp4 (792K)
1280x720 @ 30/1 25s
-- project ready: /Users/you/demo
next: cd "/Users/you/demo" && lc intake
```

(Since 1.2.0 that `next:` line prints the full path of `lc`, because `lc` is not on the PATH.)

2. Transcribe and measure. On the first run with a key, intake prints `backend: scribe` and uploads the audio (for this clip 0.8 MB and a 3 second round trip). Every later run reads the cache:

```
$ lc intake
== source
codec_name=h264
width=1280
height=720
r_frame_rate=30/1
duration=25.000000
size=810109

== transcribe (cached; costs money on the first run with Scribe)
cached: take1.json (delete it to transcribe again)

== measure audible word extents
34 words, 0 spans trimmed to their audible extent (threshold 0.00802, floor 0.00000, 60ms sustained)
the recogniser padded the END on 0, the START on 0

== transcript as readable lines
4 utterances -> transcript_lines.txt

intake done. Read transcript_lines.txt against the script, then decide the cuts.
```

`edit/transcript_lines.txt` holds one utterance per line: its span in source seconds, the silence AFTER it (99.00 on the last line, where nothing follows), and the words.

```
[ 1.58- 12.92] ( 3.36) Just do what I asked you to and then create me a final complete comprehensive
Claude project instructions so I can copy it and paste it there.
[ 16.28- 16.76] ( 1.62) ممتاز.
[ 18.38- 20.84] ( 1.98) ممتاز ممتاز. خذنا نعطيه Enter.
[ 22.82- 23.26] (99.00) تمام.
```

3. Decide the cuts (sections 6 and 7). «ممتاز» ("great") is said three times, and only the last one belongs. `lc words` shows the words around it with their logged span, their measured span and the gap before each:

```
$ lc words 12 21
12.02 12.12 | 12.02 12.12 | 0.04 | and
12.22 12.48 | 12.22 12.48 | 0.10 | paste
12.52 12.66 | 12.52 12.66 | 0.04 | it
12.72 12.92 | 12.72 12.92 | 0.06 | there.
16.28 16.76 | 16.28 16.76 | 3.36 | ممتاز.
18.38 19.00 | 18.38 19.00 | 1.62 | ممتاز.
19.06 19.46 | 19.06 19.46 | 0.06 | ممتاز.
19.52 19.74 | 19.52 19.74 | 0.06 | خلينا
19.80 20.24 | 19.80 20.24 | 0.06 | نعطيہ
20.42 20.84 | 20.42 20.84 | 0.18 | Enter.
```

Decisions name the first word removed and the first word kept after it: the first two «ممتاز» go, and the third (19.06) stays.

```
[{"rm": 16.28, "keep": 18.38, "why": "first of three «ممتاز»; the last one lands"},
 {"rm": 18.38, "keep": 19.06, "why": "second «ممتاز»"}]
```

4. Turn decisions into cuts. The two touching decisions become one cut, and the audit line shows exactly which words go:

```
$ lc mkcuts decisions.json
# 1 merged cuts from 2 decisions, 2.9s of source removed by speech cuts, 0 problems
(16.160, 19.030, "first of three «ممتاز»; the last one lands | second «ممتاز\"", 0.0, False, False),
# ...and paste it there. || [ممتاز. ممتاز. ممتاز. || خلينا نعطيہ Enter...
```

Paste the tuple whole into `SPEECH_CUTS` in `edit/cutlist.py`. The silence before the first word and after the last is not a gap between words, so it gets a head cut and a tail cut written by hand: the head ends 0.25s before the first word (1.58), the tail starts 0.4s after the last word ends (23.26).

```
SPEECH_CUTS = [
    (0.000, 1.330, "head: silence before the first word", 0.0, False, False),
    (23.660, 99999.0, "tail: after the last word", 0.0, False, False),
    (16.160, 19.030, "first of three «ممتاز»; the last one lands | second «ممتاز\"", 0.0, False, False),
]
```

5. Rebuild.

```
$ lc rebuild
-- build_edl (speech cuts only)
  2 ranges, 19.46s kept (0m19.5s) from 25.00s -- removed 5.54s
  boundary/word straddle check: 0 violation(s)
  words dropped from captions by the cut: 2 (0 NOT inside any declared cut span)
-- tighten_gaps 0.50 0.18 4.0 0.90
  4 short gap(s) + 0 long wait(s) -> 4 cuts, 4.6s removed
  2 gap(s) kept whole: too short to trim safely (under about 0.70s)
-- build_edl (with pause cuts)
  5 ranges, 14.84s kept (0m14.8s) from 25.00s -- removed 10.16s
  boundary/word straddle check: 0 violation(s)
  words dropped from captions by the cut: 2 (0 NOT inside any declared cut span)
-- build_captions
  master.ass: 8 cues (0 suppressed as duplicate of a card), font=Almarai ExtraBold @ 2560x1440
rebuild. Any SKIPPED line above is a cut that did not happen: widen it, or accept it in writing.
```

6. Check it. Each check reports what it measured:

```

$ lc post_cut_check
kept 32 of 34 words across 5 ranges
post-cut sweep clean (truncations, adjacent duplicates, n-gram repeats)
$ lc untranscribed
take1: speech p90 -20.3 dB, threshold -42.3 dB, 15s of kept audio measured
0 untranscribed audible run(s) >= 0.35s
$ lc gapcheck
delivered 0:15.0, 32 words, ceiling 2.5s
ok no wordless gap over 2.5s
$ lc waitbeats
no long waits in this cut (pause_cuts.py has no wait trims), nothing to check
$ lc capcheck
measured 8 of the widest cues: text spans x=495..783 in a 1280px frame (clearance 495px left, 497px right,
bar 24px)
$ lc head_onset_check .
take speech level: -8.7 dBFS
head OK: opens at -90.3 dB, speech -8.7 dB, rises over 160ms
tail OK: closes at -72.2 dB, speech -8.7 dB
$ lc qc transcript --edit-dir . --video /dev/null

1/1 passes OK -- READY

```

7. Read the delivered cut as prose, with every join marked:

```

$ lc delivered_read .
# DELIVERED CUT -- 5 ranges, 00:14.97, 32 of 34 words kept
[00:00.25] Just do what I asked you to and then create me a final complete comprehensive
[00:06.69] Claude project instructions so I can copy it and paste it there.
---- JOIN 01 removed 1.45s of source ( 13.00 -> 14.45) ----
---- JOIN 02 removed 4.40s of source ( 14.63 -> 19.03) ----
[00:11.93] ممتاز.
[00:12.39] خلينا نعطيه Enter.
---- JOIN 03 removed 0.84s of source ( 20.90 -> 21.74) ----
---- JOIN 04 removed 0.80s of source ( 21.92 -> 22.72) ----
...
[00:14.10] تمام.

```

One «ممتاز» is left, and it now answers the dictation directly.

8. Render.

```

$ lc render
-- pre-render: post_cut_check
clean
-- pre-render: pre-transcript
clean
-- pre-render: onset
clean
waiting for other ffmpeg process(es) to finish: 3226
-- rendering at 1280px / 30fps / CRF 18 (5 ranges, 14.8s)
a half-hour 4K lesson takes roughly an hour and a half and lands around 600 MB
-- size: 812K delivered: 15s
-- QC gate
QC GREEN
6/6 passes OK -- READY
-- HOLDING for review. Nothing uploaded.
file: /Users/you/demo/edit/final.mp4
the gate's verdict is necessary and not sufficient: read its MEASUREMENT
lines, and pull frames from /Users/you/demo/edit/final.mp4 itself at every spot you fixed.

```

The gate's measurement lines, from `edit/qc.log`, are the part worth reading:

```
. 8 cues, Almarai ExtraBold 72.0pt @ 2560x1440, MarginV 80, Outline 4.4, Shadow 0.4
. caption grid OK: master.ass accumulated 14.967s, EDL quantised to 14.967s at 30fps over 5 ranges (raw
total would have been 14.840s)
. burned captions confirmed in pixels at 4/4 sampled cues
. integrated -14.76 LUFS, true peak -0.99 dBTP
. audio/video durations agree (15.07s)
. A/V sync: audio leads by +0ms (within tolerance)
. delivered 1280x720 @ 30/1, 14.97s (EDL expects 14.84s)
. full decode clean end-to-end (no corrupt or truncated frames)
```

The render waited for another ffmpeg job on the machine to finish; that is the one-render-at-a-time rule doing its work. The delivered video is 14.97s from a 25s take.

9. Look at the file, then report and stop (section 6, "What to hand back").

6 The editorial method

This is the part that makes a lesson feel clean. The tools find candidates and place edges; the decision is always one question: does the delivered version still say everything the take said, and does it now read as one take?

What gets cut

- **A retake.** Keep the last complete attempt; it is usually the correction.
- **An abandoned attempt, back to where it began.** Cutting only the last broken word leaves the words in front of it stranded or doubled across the join.
- **A truncated word,** and a doubled word or stutter.
- **Self-talk while hunting the screen.**
- **A failed action and its retry:** keep the attempt that worked.
- **A point that lands more often than the script asks for.** The script decides how often something lands, not whether a repeat "sounds deliberate".
- **Reading something aloud that the speaker then says properly** in their own words.
- **A half-read of an on-screen line** that stops partway and goes nowhere. The screen still carries the full line.

Sweep the fillers as their own pass

A lone "OK", "right", "so", «تمام», «آآآ» or "um" between two silences is a cut. There are dozens in an hour: on the lesson this was built from, they were about a fifth of all the cuts. They are easy to read past one at a time, and together they are most of the difference between a piece that feels clean and one that feels nearly clean. A reader concentrating on structure reliably misses a third of them, so they get a pass of their own.

What never gets cut

- An idea that is not delivered again somewhere else.
- The corrections the speaker makes to the tool on camera. In a build lesson, that is the lesson.
- A parallel construction or a list that continues a sentence rather than replacing it.
- An on-screen action whose narration is the only thing explaining it.

When the speaker talks to the editor

"Delete this", «احذف هاي», «إعادة»: the instruction is a cut and so is its referent, but the scope is bounded by **re-coverage**, not by where the instruction sits. List the ideas between the previous clean point and the instruction; search forward for each one being delivered again; cut back only as far as the earliest idea that IS re-covered. Anything not re-covered stays, however rough it sounds. As the author put it: "I don't mean for everything, everything. I just mean for the last scene or the last idea that didn't really get explained correctly."

Check first that the instruction is aimed at the editor at all. The same words often mean "delete this line from the draft I am editing on screen".

A garbled delivery is cut; a garbled transcript is captioned

When a line reads as nonsense, decide which of the two it is. If it was said cleanly and the recogniser misheard, the screen shows the real words, and it is a caption correction. If the delivery itself is broken, no caption can honestly carry it, and it is a cut. Settle it mechanically: read the screen at that timestamp, and run a second transcription over the span. A second reading that matches the screen means fix the caption. Two readings that disagree with the screen and with each other mean the delivery is broken.

Which copy to keep

The one that landed. If a prompt was dictated twice, the frames decide: the take that reached the input box is the one the tool received, and keeping the other desynchronises the sound from the screen.

Silence

Not judged one gap at a time. `lc rebuild` trims every wordless gap and keeps a short beat of the busiest picture on long waits. What needs a human eye: audio with no word under it (`lc untranscribed`) and a wait beat that shows a detour instead of the work (`lc waitbeats`).

Flag, do not fix, what the picture carries

If the speaker says something the screen contradicts, or does something the script forbids, and no cut can fix it because the action is visible, cut nothing and put it in the report as the operator's decision. Cutting the audio alone would leave the screen telling a different story.

What to hand back

1. The file path and the delivered length against the raw length.
2. What was cut, grouped by kind, naming the interesting ones by timestamp.
3. What was fixed that nobody asked about (murmurs, detours, caption corrections checked against the screen).
4. The flags: things in the recording that no cut can fix, each one the operator's call.
5. Length against the script's target, with the largest trims that would lose no idea, if over.

Then stop. The operator watches it before anything ships.

7 The editorial read, with agents

A long take holds 150 to 250 real cuts. One read cannot hold them all: local stumbles need line-by-line attention, structural repeats need the whole take in view, and the two kinds of attention crowd each other out. So the read is split four ways.

Role	Agent	Finds
Window reader, one per 10 minutes	<code>lesson-window-reader</code>	Local material: attempts, truncations, doubles, fillers, self-talk, failed actions, editor instructions
Structure reader, whole take	(a general agent with the brief)	A section redone later, a point landing too often, tangents, the beat map
Adversarial verifier, one per reader	<code>lesson-cut-verifier</code>	Tries to refute each cut: content loss, wrong take, continuation, bad edges
Completeness critic, once	<code>lesson-cut-critic</code>	Applies all cuts, reads the result, finds broken joins and conflicts

Every agent reads the same brief first: `skills/lesson-cut/assets/cut-brief.md`, with its bracketed fields filled in and saved to a shared scratch directory. `skills/lesson-cut/assets/editorial-workflow.js` runs all four roles as one script for Claude Code's Workflow tool: fill its constants (project, scratch directory, take name, script path, `lc` path, duration) and pass it to that tool. Its `surviving` list uses keys `lc mkcuts` accepts directly. The script runs only through that tool. If your Claude Code does not offer it, run the same four roles as ordinary subagents, in sequence, as `skills/lesson-cut/references/editorial-read.md` describes: the structure is what matters, not the parallelism.

Three lessons from running it:

- **The verifier and the critic add cuts, not only remove them.** They contribute the last 10 to 20 percent, nearly all small fillers and signposts. Read their `missed` lists.
- **Filter findings by verdict, never by the label a reader chose.** A confirmed cut tagged `accuracy_flag` was once dropped by a filter on that tag, and only the delivered read found it.
- **Resolve alternatives explicitly.** Two versions of one cut are harmless (they are unioned). Two ALTERNATIVE fixes applied together remove both copies of a word, or leave a sentence without its verb. The critic's `wrong_cuts` and `conflicts` lists are the ones to read first.

Then the second read, after the cut is built: `lc delivered_read .` prints the cut as prose with every join marked. Hold three questions in order: does anything not make sense, does anything still need deleting, is anything repeated. A file that passes every gate can still hold things worth cutting; on one lesson this read found a batch of further cuts after a green gate.

8 Anatomy of a project

```
<project>/
├─ take1.mov          the source (or take1.mp4), moved in by lc init
└─ edit/
   ├─ project.json    render and caption settings          you (init writes it)
   ├─ cutlist.py       THE EDIT: spans of source to remove      you (from lc mkcuts)
   ├─ captions_project.py caption corrections, unbreakable phrases you
   ├─ protect.py       spans the tightener must not touch       you
   ├─ qc-accept.json   written accepts of deliberate findings   you
   ├─ pause_cuts.py    every wordless gap, trimmed              lc rebuild (tighten_gaps)
   └─ transcripts/
      ├─ take1.json     the recogniser's words                    lc intake
      └─ take1.audible.json the measured extent of every word    lc intake (measure_tokens)
   └─ transcript_lines.txt one utterance per line                lc intake (analyze_dump)
   └─ edl.json          the delivered timeline                    lc rebuild (build_edl)
   └─ master.ass        the captions                             lc rebuild (build_captions)
   └─ *.log             gate and render logs                     lc render
   └─ base.mp4          render scratch, safe to delete afterwards lc render
   └─ clips_graded/     render scratch, safe to delete afterwards lc render
   └─ qc.log            the QC gate's full report                lc render
   └─ final.mp4         the delivered file                       lc render
```

`cutlist.py` is the edit. Everything marked `lc ...` is generated, so changing a cut invalidates the EDL and the captions, and `lc rebuild` regenerates both. `lc render` refuses a `master.ass` that is older than `edl.json`.

9 Configuration reference

project.json

Written by `lc init`; edit it by hand. Every key has a default, so a missing key behaves as shown.

Key	Default	Meaning
<code>name</code>	folder name	label only
<code>language</code>	<code>auto</code>	ISO code sent to the recogniser (<code>ar</code> , <code>en</code>); <code>auto</code> detects
<code>render.scale</code>	the source's long edge (init writes it; 3840 if the key is missing)	output long edge in pixels. Never set it below the source
<code>render.fps</code>	30	output frame rate; the captions are timed for it
<code>render.crf</code>	18	18 for screen recordings, 23 for camera footage; resolution is unaffected
<code>render.av_offset_s</code>	0.0	seconds the source's audio runs AHEAD of its picture; takes effect at the next <code>lc rebuild</code>
<code>captions.profile</code>	<code>course-landscape</code>	label of the preset <code>init</code> used
<code>captions.font</code>	<code>Almarai ExtraBold</code>	font family libass must resolve
<code>captions.font_size</code>	72	size on the authoring canvas
<code>captions.outline</code>	4.4	outline width
<code>captions.shadow</code>	0.4	shadow depth
<code>captions.margin_v</code>	80	distance from the bottom edge, on the canvas
<code>captions.margin_h</code>	110	side margins, on the canvas
<code>captions.play_w</code> , <code>play_h</code>	init derives them from the source (2560x1440 if missing)	the authoring canvas; libass scales it to the real frame. <code>init</code> keeps the preset's height and takes the width from the source's aspect ratio (2560 for 16:9)
<code>captions.max_chars_per_line</code>	23	line-length target (soft)
<code>captions.max_lines</code>	2	lines per cue

The two presets `init --profile` writes:

	<code>course-landscape</code>	<code>reel-portrait</code>
Canvas	2560x1440 for 16:9	1080x1920 for 9:16
Font size, outline, shadow	72, 4.4, 0.4	66, 4.0, 1.2

	course-landscape	reel-portrait
MarginV, MarginH	80, 110	420, 90
Characters per line	23	25

FontSize is a fraction of the authoring canvas, not of the delivered frame, and MarginV 420 was calibrated to clear a phone app's bottom interface. The two sets are not convertible: portrait numbers on a landscape frame put captions in the middle of the picture, and landscape numbers on a vertical video put them under the app's buttons.

cutlist.py

```
SPEECH_CUTS = [
    (start, end, "why", keep_tail, refine_start, refine_end),
]
```

Times are source seconds. `keep_tail` keeps that many seconds of silence before speech resumes (normally 0.0); it applies only when `refine_end` is True, so it has no effect on the pinned cuts `mkcuts` writes. `refine_*` True lets `build_edl` move that edge to the nearest quiet point, right for a cut of dead air; False pins it, right for a cut that removes speech, and what `mkcuts` emits. Order does not matter: every cut is sorted and overlaps are unioned. The file also collects `PAUSE_CUTS` from `pause_cuts.py` and `EXTRA_CUTS` from an optional `extra_cuts.py`.

decisions.json (input to lc mkcuts)

A JSON array of `{"rm": <start of first word removed>, "keep": <start of first word kept after>, "why": "..."}.` A `keep` after the last word makes a tail cut to the end of the source. The editorial workflow's keys `first_removed_word_start`, `first_kept_word_start` and `reason` are accepted as well.

captions_project.py

```
CORRECTIONS = [("heard", "actual")] # substring replacements, applied per cue
NO_SPLIT = ["Claude Code", "run now"] # phrases that must stay inside one cue
STOPWORDS = {...} # optional: replaces the default (Arabic) function words
```

Corrections are substring replacements: `("pro", "pilot")` would also turn "project" into "pilotject", so make each pair long enough to be unambiguous. `NO_SPLIT` matches when a cue ends with the start of a phrase, so a phrase whose first word is common can hold cues open; keep entries specific. Write both as the recogniser spelled them. The file is executed as Python.

protect.py

`PROTECT = [(start, end), ...]` in source seconds: spans `tighten_gaps` leaves whole, for a beat where the silence is the point (a paste, a keystroke, a deliberate hold). Any overlap with a gap protects the whole gap.

qc-accept.json

A JSON array of `{"phrase": "...", "reason": "..."}` . Every entry needs a written reason; an entry without one stops the checks, and `lc render` refuses a file that does not parse. The phrase must match what the check matched. The gate compares after normalising (diacritics, hamza forms, case and punctuation are ignored); `post_cut_check` compares the text as transcribed:

Finding	Phrase
Truncated word	the token exactly as transcribed, dashes included
Adjacent duplicate	the two tokens exactly, punctuation included, one space between
Immediate two-word restart	the two-word phrase exactly
N-gram repeat	the repeated run as transcribed; at least its first two words must match
Restart across a pause	the words the finding prints, in its order
Black bar on an edge	<code>black bar on left edge</code> or <code>black bar on top edge</code>

Nothing else can be accepted: a range edge inside an audible word, the caption checks, loudness, dead air, sync, duration and resolution can only be fixed.

Environment variables

Variable	Meaning
<code>ELEVENLABS_API_KEY</code>	the Scribe key; wins over the file
<code>LESSON_CUT_HOME</code>	where the venv, the key file and the render lock live (default <code>~/.lesson-cut</code>)
<code>LC_PYTHON</code>	force a specific python instead of the venv
<code>ACCEPT_FINDINGS=1</code>	same as <code>lc render --accept-findings</code>

`lc` sets `LC_EDIT_DIR`, `PYTHONPATH` and `PYTHONDONTWRITEBYTECODE` itself for every tool it runs.

10 Command reference

Every command runs from the project directory or its `edit/` directory. `--project <dir>` anywhere on the line points at another project. A relative file path is read from where you stand; `.` means the edit directory.

`lc doctor [font]` : checks the machine (section 3). Exits 1 when something is missing.

`lc init <dir> --source <video> [options]` : creates the project. Options: `--profile` `course-landscape|reel-portrait`, `--lang <code>`, `--scale <px>` (default: the source's long edge), `--fps <n>` (30), `--crf <n>` (18), `--copy` (copy instead of move). Refuses to overwrite an existing project. Sources must be `.mov` or `.mp4`.

lc intake [--backend auto|scribe|local] [--lang <code>] [--model <size>]: probes the source, transcribes it (cached), measures every word, writes transcript_lines.txt. auto uses Scribe when a key exists and local whisper otherwise, and says which. --model is the whisper size (default large-v3).

lc rebuild [short_ceiling short_target long_ceiling long_target] [--no-captions]: empties pause_cuts.py, builds the EDL from the speech cuts, tightens every gap against it, builds the EDL again, builds the captions. Defaults 0.50 0.18 4.0 0.90. Stops at the first failure.

lc render [--scale px] [--fps n] [--out file.mp4] [--accept-findings]: refuses a missing, stale or mis-timed master.ass, a project.json that does not parse and a malformed qc-accept.json; runs three gates; takes the render lock and waits for other ffmpeg processes (and says which); renders to a temporary file; runs the six-pass QC gate, then post_cut_check and head_onset_check once more; and stops. Exits 1 on any refusal or failure. --accept-findings renders past reported findings, never past a gate that crashed, and the verdict still says NOT CLEAN while a finding stands. --out is read from where you stand, must be a .mp4 or .mov, and is never allowed to replace a source or any existing file outside edit/.

lc transcribe <video> --edit-dir . [--backend ...] [--language ...] [--num-speakers n]: transcription on its own; lc intake normally calls it.

lc qc <all|transcript|captions|caption-px|video|audio|spec> --edit-dir . --video <file>: the QC gate, or one pass of it. transcript needs no video: --video /dev/null.

lc tools: lists the commands and the tools callable as lc <name>.

lc <tool> [args]: runs any tool in section 11.

11 Tool reference

Each tool runs through `lc`. "Reads" and "writes" are relative to `edit/`.

Intake and measurement

measure_tokens · `lc measure_tokens`. Measures the audible extent of every word from the audio and writes `transcripts/take1.audible.json`. Spans only ever shrink. Long tokens use the longest sustained run of sound, so one stray loud frame inside a padded pause cannot stretch a word across it. Re-run after any change to `take1.json`: the two files are matched by position.

analyze · `lc analyze <dump|tokens|deadair|instr|repeats|latin> [args]`. Scans of the raw transcript. `dump` is what `transcript_lines.txt` contains. `tokens` lists words the recogniser stretched across silence. `deadair` lists quiet runs. `instr` finds spoken editor instructions (substring, case-sensitive). `repeats` finds n-gram repeats. `latin` lists Latin-script tokens to check against the screen.

words · `lc words <from_s> <to_s>`. Words in a window: logged span, measured span, gap before, text. Take 1 only.

picture · `lc picture <window|sweep|inset> [args]`. Per-second screen activity: how many of the 3600 blocks (8x8 pixels each) of a 640x360 frame changed. `window a b` classifies one window; `sweep` classifies `start end` rows from stdin (the output of `lc analyze deadair` works as is); `inset` looks for a webcam circle. The analysis frame masks a small bottom-left box as a webcam inset by default (see section 15).

transcribe, transcribe_local · see `lc intake`.

Authoring the cut

mkcuts · `lc mkcuts decisions.json`. Decisions to cut-list tuples, unioned by word index, edges padded by `min(0.03, gap/2)`, head cut 0.25s before the first kept word, tail cut to the end. Prints an audit line per cut showing the dropped words. Exits 1 if a decision has keep before rm, rm past the last word, or removes every word. It works in words: to trim only the silence before the first word or after the last, write the head or tail cut by hand (section 5 shows both).

build_edl · `lc build_edl [--write]`. Builds `edl.json` from `cutlist.py`. For each cut it refines unpinned edges to quiet points, snaps every edge out of words, and checks the result. Reports straddles (an edge inside a word). REFUSES to write when a word is dropped that no declared cut covers. Prints `SKIPPED` for a cut whose refined width fell under 0.15s: that cut did not happen. Stamps `av_offset_s` from `project.json`, `subtitles: master.ass`, and the total duration. Needs `take1.audible.json`.

tighten_gaps · run through `lc rebuild`. Trims every wordless gap on the delivered timeline, measured across joins, into `pause_cuts.py`. A gap over the short ceiling keeps about the short target from its middle; a wait over the long ceiling keeps the long target of its busiest picture. A gap under about 0.7s (at the defaults) is kept whole and counted. The inside of every cut is measured too: a cut whose interior holds speech-level sound that no transcript word accounts for (0.2s of it in any half second, which a short word reaches and a click does not) is not made, because the transcript probably missed a word there, and it is reported. Sound within a word, or up to 0.8s after one, belongs to that word: a removed filler rings on well past its measured end, and trimming that ring is what the cut is for. Refuses to run when `pause_cuts.py` already has cuts, because it would regenerate an empty list.

merge_edl · `lc merge_edl edl-take1.json edl-take2.json [--write]` . Joins per-take EDLs of a stitched lesson into `edl.json` , in argument order.

tighten · `lc tighten [ceiling]` . An older silence-ceiling enforcer on the assembled audio (sees 2s and longer only). Secondary; single take.

Checks

post_cut_check · `lc post_cut_check` . On the delivered word sequence: truncated words, adjacent duplicates, immediate two-word restarts, n-gram repeats (the gate's windows). Exits 1 on a finding. Honors `qc-accept.json` .

untranscribed · `lc untranscribed [min_seconds]` . Speech-level audio inside kept ranges that no word covers (default 0.35s). Advisory: exits 0; judge each hit.

waitbeats · `lc waitbeats` . A contact sheet per eight kept wait beats (before, beat, after), written as `waitbeats_NN.jpg` . Says so and exits 0 when the cut has no long waits.

gapcheck · `lc gapcheck [ceiling] [--emit]` . Wordless gaps on the delivered timeline, word to word, no audio threshold (default ceiling 2.5s): between kept words, before the first and after the last. `--emit` prints cuts for the gaps it fails.

head_onset_check · `lc head_onset_check` . The first and last 1.2s of the cut against the take's speech level: the head and tail must not clip a word. Can miss a soft final word; listen too.

capcheck · `lc capcheck [n]` . Renders the widest n cues (default 60) through libass on the delivered frame and reports how close they come to the edges.

delivered_silence · `lc delivered_silence [min_run] [max]` . Dead air on the assembled audio before loudness normalisation. Secondary; take 1 only.

Captions and reading

build_captions · `lc build_captions` . Writes `master.ass` from the EDL and the measured transcript: cues never cross a cut, break at sentence ends, at the character limit or at a 0.55s pause, last at most 7s and at least 0.55s unless the next cue starts sooner, never end on an Arabic article such as «`ﻟﻰ`», keep `NO_SPLIT` phrases on one line, and are timed on the renderer's whole-frame grid for `render.fps` (stamped into the file). A line that contains right-to-left script, such as Arabic, is wrapped in right-to-left embedding, so a Latin word inside it cannot scramble the order of the Arabic around it. A line with no right-to-left letters (an English take, or an all-Latin line in an Arabic one) is left alone, because inside that embedding a leading number or edge punctuation would jump to the wrong end of the line.

delivered_read · `lc delivered_read` . The delivered cut as prose with every join marked and what it removed, timed on the renderer's frame grid. Raw recogniser text, no caption corrections.

Render and gate

render (`lc render` , see section 10). Extracts each range with 30 ms audio fades, concatenates losslessly, then burns the captions in a final pass at `render.crf` with two-pass loudness normalisation to -14 LUFS. The renderer itself, `scripts/render.py` , also has `--draft` (1280 long edge, fast) and `--preview` modes, reachable by calling it with the venv's python; always pass `--fps` equal to `render.fps` .

qc · the six-pass gate, section 12.

grade · `lc grade --analyze <video>` . Colour-grade analysis. The lesson pipeline grades nothing by default; auto grading is tuned for camera footage and lifts a dark screen recording.

12 The checks, and what each one alone can see

Defect	Caught by
A cut edge inside a word	<code>build_edl</code> (reports), <code>qc transcript</code> (fails when audible)
A word dropped outside any declared cut	<code>build_edl</code> (refuses to write)
A cut that silently did not happen	<code>build_edl</code> prints SKIPPED
A truncated word that survived	<code>post_cut_check</code> , <code>qc transcript</code>
The same word twice, touching	<code>post_cut_check</code> (the gate does not look for this)
A two-word phrase said twice with no pause between	<code>post_cut_check</code>
A restart after a pause in the take, reworded or cut short	<code>qc transcript</code>
A repeated 3 to 6 word run	<code>post_cut_check</code> , <code>qc transcript</code>
A flash of picture between two cuts	<code>build_edl</code> drops wordless slivers under 0.1s
Sound with no word under it	<code>untranscribed</code> (reports); <code>tighten_gaps</code> (will not trim it away)
A wait beat showing a detour	<code>waitbeats</code> only (a person looks)
Dead air between words	<code>gapcheck</code> ; the QC audio pass on the render
A clipped first or last word	<code>head_onset_check</code>
A caption touching an edge	<code>capcheck</code> ; QC caption-px
Captions timed for another fps, or for an older cut	<code>lc render</code> refuses; QC captions
No captions in the delivered pixels	QC caption-px, which reads the finished file
Output smaller than the source	QC spec
Wrong loudness, clipping	QC audio
Lip sync	a person, by eye (the gate compares stream start times only)
Something private on screen	a person, the visibility sweep

The QC gate's six passes

1. **transcript**: truncations, range edges inside audible words, restarts across a pause the speaker made in the take (measured in the take, so the tightener's trimming does not hide it), n-gram repeats. Needs no video, so it also runs before the render.

2. **captions (data)**: the caption grid matches the render's fps and the cut; no stray `--`, orphan letters, trailing punctuation, right-to-left embedding missing from a line with right-to-left script or present on a line without any, overlaps, cues past the end, more than two lines, or a margin under 40.
3. **captions (pixels)**: renders every cue through libass and checks the edges, then reads the delivered file to confirm captions are actually burned in.
4. **video**: near-black frames and black bars on the left or top edge; freezes are noted, not failed.
5. **audio**: loudness within -16.5 to -11.5 LUFS, no clipping, no dead air (every silence of 2s or more is judged, and fails only when the screen was also still, because on a screen recording watching the tool work is content), the audio and the video STREAM the same length, and start times in sync.
6. **spec**: the video stream's duration matches the EDL, the output is not smaller than any source, and the whole file decodes cleanly.

The gate prints `N/6 passes OK -- READY` or `-- NOT READY`, and the full report is `edit/qc.log`. Read its measurement lines, not just the verdict.

13 Troubleshooting

Symptom	Cause	Fix
<code>lc: no project here</code>	not in a project directory	<code>cd</code> into the project, or <code>--project <dir></code>
<code>no transcript in this project yet</code>	intake not run	<code>lc intake</code>
<code>no ElevenLabs key, and no local ASR</code>	no transcription backend	add a key, or <code>setup.sh --local-asr</code>
<code>Scribe</code> returned 401	key rejected	check the key in <code>~/.lesson-cut/env</code>
doctor: <code>font ... fell back to</code>	the caption font is not installed	install it, or set <code>captions.font</code>
A cut in the list had no effect	it printed SKIPPED	paste <code>mkcuts'</code> tuple whole (pinned edges), or widen it
Dead air came back after a rebuild	tightener ran over pause cuts (old versions)	always use <code>lc rebuild</code>
<code>tighten_gaps</code> refuses	<code>pause_cuts.py</code> is not empty	<code>lc rebuild</code>
<code>lc render</code> : <code>master.ass</code> is older than <code>edl.json</code>	the cut changed after the captions	<code>lc rebuild</code> or <code>lc build_captions .</code>
<code>lc render</code> : built for 25fps, render is 30fps	fps changed	set <code>render.fps</code> , rebuild captions
REFUSING TO RENDER	a gate found something	fix it, or accept it in writing, then render
QC: NO burned captions found	<code>subtitles</code> missing in the EDL	<code>lc rebuild</code> (it sets it)
QC: OUTPUT ... SMALLER THAN SOURCE	<code>render.scale</code> below the source	set it to the long edge
Captions drift progressively early	captions built for another fps	rebuild them at <code>render.fps</code>
QC: line(s) with no right-to-left script are wrapped in RTL embedding	<code>master.ass</code> was built before 1.2.0, or edited by hand	<code>lc build_captions .</code> , then render
Lips ahead of or behind the voice	<code>render.av_offset_s</code> wrong for this rig	measure it, set it, <code>lc rebuild</code> , render
<code>lc render</code> waits a long time	another render or ffmpeg is running	by design; it prints the PIDs it waits for

Symptom	Cause	Fix
rebuild: <code>cut(s)</code> NOT made: the gap holds speech-level audio	the transcript missed a word, or a murmur	listen; keep it, or cut it on purpose as a speech cut
<code>REFUSING TO RENDER: ...</code> crashed	a gate hit a tool error, not a finding	read the named log; <code>--accept-findings</code> does not apply
verdict ends <code>NOT CLEAN</code>	rendered with <code>--accept-findings</code> past a finding	accept it in <code>qc-accept.json</code> with a reason, or fix it
<code>render: project.json</code> does not parse	a JSON syntax error	fix the file; defaults are never guessed
doctor: <code>ffmpeg</code> has no libass	a plain <code>ffmpeg</code> build (Homebrew's <code>ffmpeg</code>)	<code>brew install ffmpeg-full</code> , then <code>brew unlink ffmpeg</code> && <code>brew link ffmpeg-full</code>
<code>render: ...</code> already exists outside <code>edit/</code>	<code>--out</code> points at an existing file, such as the take	choose a new name; the take is never overwritten
rebuild: this is a stitched (multi-take) lesson	more than one take in the project	build each take with <code>build_edl --source</code> , then <code>merge_edl</code>
<code>render: qc-accept.json</code> must be ...	an accept entry without a phrase or a reason	fix the entry; every accept needs both
dropped <code>N</code> kept sliver(s) under <code>0.10s</code>	two cuts left a flash of picture between them	nothing to do; it is removed on purpose
The opening or ending has seconds of silence	<code>tighten_gaps</code> only trims between words	add a head or a tail cut (section 5)
The plugin stopped working after moving your clone	a folder marketplace points at the old path	re-run <code>/plugin marketplace add <new path></code>
Cannot add marketplace "lesson-cut" when adding the GitHub one	an older folder install is still registered under that name	<code>/plugin marketplace remove lesson-cut</code> , then add and install again (section 3)
An update did not arrive	<code>/plugin marketplace update</code> alone does not update the plugin, and an unchanged version is not reinstalled	<code>/plugin update lesson-cut@lesson-cut</code> , then restart Claude Code
words: <code>N</code> but a different count elsewhere	the recogniser also returns spacing tokens	the tools count words only

14 Operating contract for agents

An agent operating this plugin follows these rules. They are short because each one was learned the expensive way.

1. Run `lc doctor` before the first edit on a machine, and fix what it reports.
2. Call `lc` by its full path in every shell command.
3. Never transcribe twice. The cache is the transcript file; delete it only on purpose.
4. Move the source, never copy it, unless told.
5. Read the script before cutting, and cut against it.
6. Do the editorial read in four passes, and sweep the fillers as their own pass.
7. Express every cut through `lc mkcuts` and paste its tuples whole; only the head and tail cuts are written by hand.
8. After any change to the cut list, `lc rebuild`, and read every `SKIPPED` line.
9. Run every check in section 12's list before the render, and read the delivered cut as prose.
10. An accept is a written judgement with a reason, never a loosened threshold.
11. Deliver only what `lc render` produced (a `render.py --draft` is for looking, never for delivery). Never lower the resolution; raise the CRF if size matters.
12. Check lip sync by eye once per recording setup.
13. After the gate, look at frames from the finished file yourself.
14. Flag what the picture carries; do not cut the audio out from under it.
15. Never upload or publish unless the operator says so. Stop at the report.
16. Every correction the operator gives becomes a written rule the same session.

15 Limits and known gaps

Stated plainly, so nobody finds them the hard way.

- **Single take is the main path.** Stitched two-take lessons are supported by `build_edl --source take2` and `merge_edl`, but `intake`, `rebuild` (it refuses a stitched lesson), `words`, `mkcuts`, `tighten_gaps`, `waitbeats`, `head_onset_check` and `delivered_silence` assume take 1.
- **Webcam masks are fixed.** The picture analysis (`picture`, and through it the tightener's choice of "busiest" frame) and the QC dead-air check both ignore a small bottom-left box, placed for a webcam inset. On footage with no webcam there, activity in that corner is not counted; on footage with a webcam elsewhere, face movement can look like screen activity.
- **Local transcription is weaker** on product names, mixed languages and dialect, and the tiny and base models are not usable for Arabic (section 3). Check Latin tokens against the screen.
- **Built on Arabic lessons.** The defaults (the caption font, caption rules such as never ending a cue on «J|», and the function-word lists the checks ignore) were tuned on Arabic lessons with English terms in them. Other languages go through the same pipeline, but no long take in another language has been run; set a caption font that covers your script, and look at a rendered frame.
- **Lead-in and tail silence are trimmed by hand**, with a head and a tail cut; the tightener only works between words.
- **The transcript cache is keyed on the file name.** A different recording under the same name reuses the old transcript until you delete it.

- **Caption line length is a target.** A line can run a few characters over; the pixel checks are what guard the edges.
- **The tightener leaves gaps under about 0.7s whole** at the default settings, and says how many.
- `head_onset_check` **can miss a soft final word.** Listen to the last second.
- **Verified on macOS.** Linux is expected to work and has not been run. Windows is not supported outside WSL.
- **No upload step.** By design.
- **Comments in the tools cite rule numbers** from the author's private ledger, which does not ship. The comment itself carries the reason.
- **Known and left for later**, each small and each with a safe path around it: emptying `pause_cuts.py` by hand and running `tighten_gaps` directly can still restore dead air (use `lc rebuild`); the stale-caption check compares whole-second file times (`lc rebuild` always rebuilds captions after the EDL); a failed `init --copy` with an invalid number leaves the copied take behind; the doctor accepts a placeholder key, which then fails at transcription with a clear message; a value flag given no value to `render` or `intake` prints a raw shell error; `intake` prints a traceback for a `project.json` that does not parse; local-whisper words sharing one timestamp can overlap by a few milliseconds.

16 How it was verified

Before its public release, the plugin went through two rounds of testing by independent agents that could not see how it was built, each in its own sandbox, each working from a clean copy of the repository (built with `git archive`). A finding counted only after a second, adversarial agent re-ran it and confirmed it.

Round one, on version 1.0.1: eight testers (install, local transcription, paid transcription, edge cases, a regression on the real lesson the plugin came from, a docs audit, a code review, a privacy sweep), each paired with a verifier, and six readers who read and ran every tool to write chapter 11. The testers reported 106 findings. The verifiers re-ran every one of them: all 106 were confirmed, and none was refuted.

Round two, on version 1.1.0: three agents re-ran all 106 round-one findings against the new version (79 fixed, 21 partly fixed, 3 still present, 3 no longer applicable because the tool was removed), a newcomer followed the README and this guide literally, two agents fact-checked every sentence of this guide, one re-swept for privacy, and one reviewed the fixes themselves for new bugs. They reported 73 new findings, most of them a few defects found by several agents at once. The verifiers confirmed 71 of the 73, refuted one, and could not settle one.

Then **one consolidated correction round** fixed everything that could lose data, ship a wrong file without saying so, break a documented step, or make this guide wrong. What was left is small, has a safe way around it, and is listed at the end of chapter 15. A final paired gate, on the 1.1.1 code, confirmed the result:

- **The real lesson.** From its shipped cut list, the EDL matches the shipped one except for a couple of dozen kept slivers under 0.1s (about two seconds in total), which are now removed on purpose. On the same raw EDL, the tightener makes exactly the same cuts as the author's own production tightener, cut for cut, while still protecting a word the transcript misses. The gate's transcript pass is READY.
- **A fresh install.** From a clean copy: setup, init, intake, the worked example in chapter 5, every check and the render, 6/6. The raw take was refused as a render output, and a file with half its video missing failed the gate (4/6).

Across both rounds:

- **Install:** through Claude Code's plugin commands, from a folder marketplace, into an isolated configuration; `claude plugin validate --strict` passes on every component. Both installs in chapter 3 that start from GitHub (the marketplace and the clone) are new in 1.2.0 and were not part of these rounds.
- **Transcription:** one real ElevenLabs call on a 25s bilingual clip, the cache proven on a second run, the no-key paths checked; local whisper run end to end, with the model measurements in chapter 3.
- **Edge cases:** paths with spaces, Arabic and apostrophes, upper-case extensions, a symlinked `lc`, macOS's bash 3.2, missing inputs, stale captions, fps mismatches, portrait takes.

What neither round ran: Linux, Windows under WSL, and the installs from GitHub above.

The campaign found and fixed, among others: a 0.24s lip-sync offset from the author's own rig stamped into every EDL; portrait takes rendering at 608x1080; a render option that could delete the raw take; a gate that passed a file with half its video missing; a restart check that the tightener's trimming had silenced; two-frame flashes of picture in a shipped lesson; a tightener that could restore all the dead air, or delete a word the transcript missed; a font check that passed every font, and a doctor that said "ready" on an ffmpeg that cannot burn captions.

Version 1.2.0, the public release, went through a round of its own. Five independent auditors read the whole repository, each for one thing: exposure of private details, licensing, the GitHub install, the public documentation, and repository hygiene. A skeptic re-checked every finding, and a critic looked for what all five had missed. 67 findings were confirmed and one was refuted. The fixes were made in six groups of files, each reviewed by its own skeptic and repaired once. Then:

- **The GitHub installs**, in an isolated configuration, against a git-served copy of this repository (the public repository did not exist yet): the marketplace add, the install, an update across a version bump, and the move from a folder install.
- **Captions**. On the same real Arabic lesson, the new builder produced the same cues with the same timings as 1.1.1. Every line containing Arabic was byte-identical, and the only change was that the all-Latin lines lost their right-to-left embedding. The caption data pass was READY on that lesson's frame grid, and `lc render` refused the lesson's 1.1.1 captions before rendering. An English clip now renders "10 minutes later" in the right order: rebuilt and rendered end to end on 1.2.0, it passed every gate and the QC gate 6/6, and its caption was read off a frame of the finished file.
- **Setup**, under bash 3.2: a virtualenv left without pip is rebuilt, the uv route works, and a `venv` folder that is not a virtualenv is refused with its files left untouched.
- **Static checks**: every shell script parsed by bash 3.2, every Python file compiled, both JavaScript files checked, `claude plugin validate --strict` passing, and every shipped file scanned for private details and invisible characters.

Not run for 1.2.0: Linux, Windows under WSL, and a full 4K render of a long lesson (the render code itself did not change).

17 Maintaining and sharing it

Versions follow MAJOR.MINOR.PATCH in `.claude-plugin/plugin.json`; `metadata.version` in `.claude-plugin/marketplace.json` and line 3 of this guide agree with it. Claude Code keeps installed copies by version, so a change reaches people who installed the plugin only when the version number changes. Updating an install is in section 3; `CONTRIBUTING.md` lists the release steps, and `docs/build/README.md` says how to rebuild the PDF of this guide.

Changing a tool: read its whole file first; most comments name the failure that put a line there. After a change, rebuild a real project and compare its `ed1.json` and `master.ass` with the previous version's before trusting it. A change that makes a check stricter is safer than one that makes it quieter.

Sharing: point people at the repository, <https://github.com/aiwithmo/lesson-cut>. It is the copy that gets updates, and section 3 installs from it.

Reporting a problem: open an issue on the repository, with the `lc doctor` output (`CONTRIBUTING.md` says what else helps). Never paste an API key into an issue. A security problem goes through the repository's private vulnerability reporting (its Security tab), not a public issue; `SECURITY.md` has the details.

Relation to the author's own pipeline: this plugin was extracted from a private production pipeline and is maintained as a fork, not a mirror. A fix in one does not reach the other by itself.

License: MIT, in `LICENSE` . Three scripts derive from video-use, also under the MIT License, and a fourth reuses one helper from it; its notice ships in `THIRD_PARTY_NOTICES.md` .

18 Glossary

Term	Meaning
take	the raw recording, <code>take1.mov</code>
range	a kept span of the source; the EDL is a list of them
EDL	edit decision list: <code>edl.json</code> , the delivered timeline
join	where two ranges meet in the delivered file
cut	a span of source removed; declared in <code>cutlist.py</code>
speech cut	a cut that removes words; its edges are pinned
pause cut	a trim of a wordless gap, generated into <code>pause_cuts.py</code>
logged span	a word's start and end as the recogniser reported them
measured span	a word's start and end as <code>measure_tokens</code> measured them from the audio
re-coverage	whether an idea removed by a cut is delivered again later
wait beat	the short slice of a long wait kept so the viewer sees the tool working
gate	a check that refuses to let the next step run
accept	a written, reasoned exception in <code>qc-accept.json</code>
canvas	the caption authoring size (PlayRes); libass scales it to the frame
CRF	the encoder's quality setting; higher means smaller, never lower resolution